

Sorting Algorithms in Data Structures

Sorting rearranges elements in a collection into a specific chronological or numerical order. Mastering sorting algorithms is essential for optimizing searching, data analysis, and algorithm design problems heavily featured in data structure exams.

Definition and Classification

- Sorting transforms unsorted input data into a monotonically non-decreasing or non-increasing sequence.
- Internal sorting assumes all data fits entirely in main memory (RAM), whereas external sorting handles massive datasets using external storage like hard drives.
- In-place sorting requires only a constant amount $O(1)$ of extra memory space beyond the input array.
- Adaptive sorting algorithms take advantage of existing order in the input, running faster on partially sorted datasets.

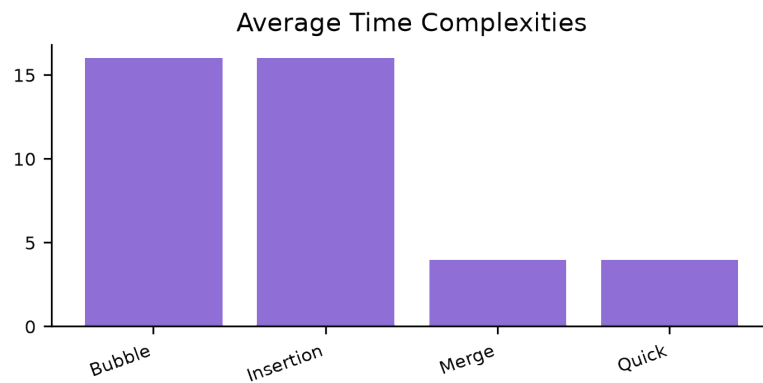
Stability and Key Properties

- A sorting algorithm is stable if it preserves the relative order of identical elements (keys) from the original input.
- Stability is crucial when sorting records by multiple criteria, such as sorting students by grade and then sorting the result by name.
- Comparison-based sorting algorithms rely solely on comparison operators (less than, greater than) to determine relative order.
- Non-comparison sorts use the internal structure of keys, such as digits or characters, to achieve linear time complexity.

Comparison Sorting Time Complexities

- Comparison sorts possess a theoretical lower bound of $O(n \log n)$ time complexity based on decision tree models.
- Bubble, Insertion, and Selection sorts operate in quadratic $O(n^2)$ time on average and worst-case scenarios.
- Merge sort and Heap sort guarantee worst-case $O(n \log n)$ performance, making them dependable for large datasets.
- Quicksort averages $O(n \log n)$ time, though its worst-case is $O(n^2)$ unless optimized with randomized pivots or median-of-three selection.

$$O(n \log n)$$



Merge Sort Execution Flow

- Merge sort is a classic divide-and-conquer algorithm that systematically splits data, sorts, and merges halves.
- It consistently achieves $O(n \log n)$ time complexity in all cases, unlike Quicksort.
- It requires $O(n)$ auxiliary space for the temporary arrays used during the merge step, making it not strictly in-place.
- It is naturally stable and highly effective for linked lists due to sequential access patterns.

Merge Sort Process



Non-Comparison Sorting

- Counting sort counts the frequency of each distinct key value, achieving $O(n + k)$ time where k is the range of input.
- Radix sort processes keys digit by digit from least significant to most significant using a stable sorting subroutine.
- Bucket sort distributes elements into a number of buckets, which are then sorted individually using insertion sort or recursively.
- Non-comparison sorts bypass the $O(n \log n)$ lower bound restriction by avoiding direct element comparisons.

$$O(n + k)$$

Key Terms

Stability	The property of a sorting algorithm that maintains the relative input order of duplicate elements.
In-Place	An algorithm characteristic requiring only a constant amount of extra auxiliary memory $O(1)$.
Decision Tree	A binary tree model used to prove the $O(n \log n)$ lower bound for comparison-based sorting.
Pivot	A chosen element in partitioning algorithms like Quicksort used to divide the array into smaller sub-arrays.
Adaptive	An algorithm property where performance improves when the input is already partially sorted.
External Sorting	A class of sorting algorithms that can handle massive amounts of data exceeding main memory capacity.

Exam Tips

- ★ Memorize the time and space complexities (Best, Average, Worst) for Bubble, Selection, Insertion, Merge, Quick, and Heap sorts.
- ★ Be ready to trace at least two passes of Quicksort partitioning or Merge Sort splitting/merging on an exam question.
- ★ Remember that Merge Sort is stable and $O(n \log n)$ worst-case, but requires $O(n)$ extra space.
- ★ Know the distinct difference between comparison sorts and non-comparison sorts (Counting, Radix) regarding the $O(n \log n)$ barrier.